AWS EKS: Kubernetes

Armanjot Singh
Diego Miguel
Sergio Morales
Hans Jeremi

ÍNDICE

Introducción a EKS **3**
PROYECTO Y OBJETIVO GENERAL **5**
CONCEPTOS FUNDAMENTALES DE KUBERNETES APLICADOS A NUESTRO PROYECTO **5**
Contenedores e imágenes 5
Pod como unidad mínima de ejecución 6
Nodos y clúster 6
Control plane y reconciliación del estado 6
Deployment y ReplicaSet 7
Namespace como segmentación lógica 7
RED EN KUBERNETES Y CONECTIVIDAD ENTRE COMPONENTES DEL PROYECTO .. **8**
Service discovery y por qué no usamos IPs de Pod 8
TIPOS DE SERVICIOS Y QUÉ UTILIZAREMOS EN EL PROYECTO **8**
ClusterIP como servicio interno 8
NodePort como forma de exponer pero menos profesional 9
LoadBalancer como entrada profesional a la aplicación 9
PERSISTENCIA Y DATOS EN KUBERNETES ORIENTADO AL PROYECTO WORDPRESS MYSQL **10**
Por qué la persistencia es obligatoria en nuestro caso 10
EBS como backend de volúmenes en AWS **10**
EFS como alternativa teórica y cuándo tendría sentido 11
SEGURIDAD Y CONFIGURACIÓN CON SECRETS EN EL PROYECTO **11**
Secrets y por qué no ponemos contraseñas en plano 11
OBSERVABILIDAD Y TROUBLESHOOTING QUE USAREMOS **12**
Kubectl como herramienta principal de operación 12
Describe y logs como diagnóstico 12
EKS CLOUD Y HERRAMIENTAS AWS QUE APORTAN CONTEXTO REAL AL PROYECTO **13**
Credenciales temporales y Cloud9 13
Kubeconfig y contexto del clúster **13**
Addons y drivers CSI para EBS 13
FLUJO DE DESPLIEGUE ORIENTADO A LA MEMORIA DEL PROYECTO **14**
Preparación del clúster 14
Despliegue de la capa de datos MySQL 14
Despliegue de la capa de aplicación WordPress 14
Validaciones y demostraciones que aportan valor 15
PROS/CONTRAS DEL ENFOQUE Y CÓMO LOS JUSTIFICAMOS EN EL PROYECTO ... **15**
Ventajas principales 15
Limitaciones y decisiones conscientes 15
CONCLUSIÓN CON ENFOQUE TOTAL AL PROYECTO **16**
WEBGRAFÍA **18**

INTRODUCCIÓN A EKS

Amazon Elastic Kubernetes Service (EKS) es un servicio de Kubernetes completamente administrado que elimina la complejidad que supone operar clústeres de Kubernetes.

Con EKS, es posible:

- Implementar aplicaciones más rápido y con menos sobrecarga operativa
- Escalar sin interrupciones para satisfacer las demandas cambiantes de las cargas de trabajo
- Mejorar la seguridad mediante la integración con AWS y actualizaciones automatizadas
- Puede elegir entre EKS estándar o el modo automático de EKS totalmente automatizado .

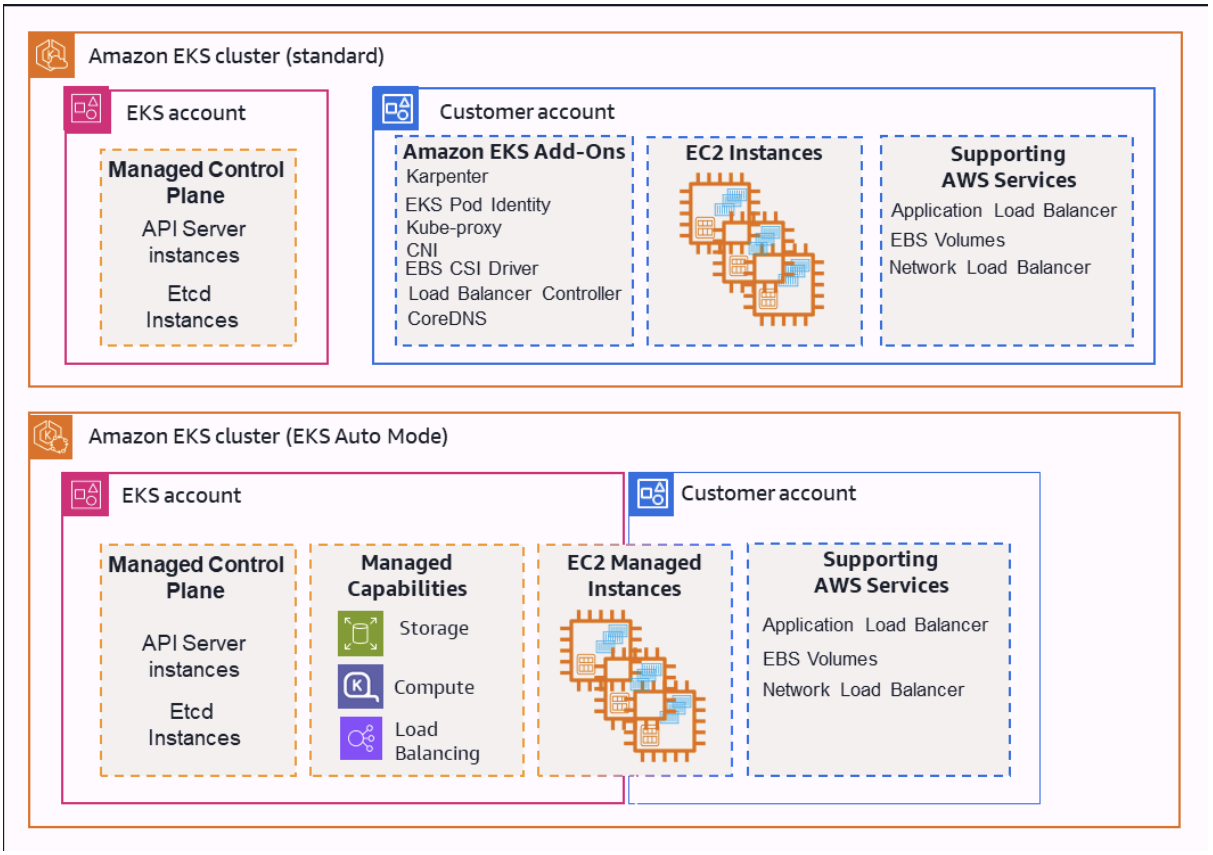
La principal ventaja de EKS radica en su modelo de responsabilidad compartida, que divide la arquitectura en dos grandes bloques:

- El Control Plane (Gestionado por AWS): AWS se encarga de aprovisionar, escalar y administrar automáticamente la infraestructura del plano de control en múltiples Zonas de Disponibilidad (AZs) para garantizar su resiliencia. Como administradores, nosotros no interactuamos directamente con estos servidores subyacentes ni nos preocupamos por su mantenimiento; simplemente consumimos la API de Kubernetes.
- El Data Plane o Nodos de Trabajo (Gestionado por el equipo): Nuestra responsabilidad se centra en configurar y administrar los *Worker Nodes* (las instancias Amazon EC2 donde realmente se ejecutan los contenedores de nuestra aplicación). Para este proyecto, utilizamos la característica de Managed Node Groups a través de la consola gráfica de AWS, lo que nos permite definir fácilmente la capacidad de cómputo y el autoescalado de nuestros nodos.

Amazon EKS facilita la creación, protección y administración de clústeres de Kubernetes. Puede resultar más rentable proporcionar los recursos necesarios durante los picos de demanda que mantener centros de datos propios. Estas son dos de las principales formas de utilizar Amazon EKS: EKS estándar: AWS administra el plano de control de Kubernetes cuando se crea un clúster con EKS.

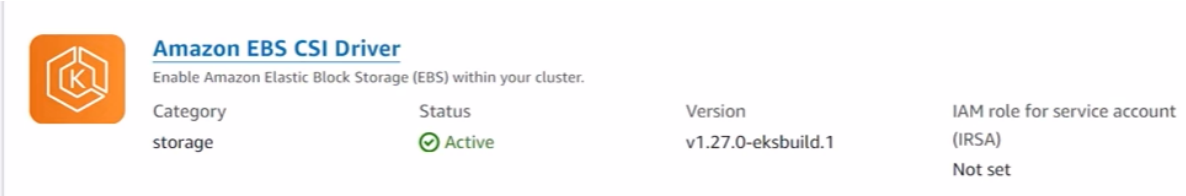
Los componentes que administran los nodos, programan las cargas de trabajo, se integran con la nube de AWS y almacenan y escalan la información del plano de control para mantener los clústeres en funcionamiento se administran automáticamente en su nombre. Modo automático de EKS: cuando se utiliza la característica Modo automático de EKS, EKS amplía su control para administrar también los nodos (plano de datos de Kubernetes). Simplifica la administración de Kubernetes al aprovisionar automáticamente la infraestructura, seleccionar las instancias de computación óptimas, escalar dinámicamente los recursos, optimizar continuamente los costos, aplicar revisiones a los sistemas operativos e integrarse con los servicios de seguridad de AWS.

El siguiente diagrama ilustra cómo Amazon EKS integra los clústeres de Kubernetes con la nube de AWS, según el método de creación de clúster que elija:



Despliegue EKS: WordPress CON MySQL

Para realizar este despliegue primeramente en el cluster debemos agregar los drivers en add-on, en particular el siguiente:



En este laboratorio, el objetivo es desplegar una página web completa con WordPress y su base de datos MySQL, todo gestionado dentro de un clúster de Kubernetes en Amazon (AWS EKS). Lo más interesante de esta práctica es que hace mucho hincapié en cómo no perder la información si algo falla, utilizando discos de almacenamiento persistente.

Para montar todo esto, la arquitectura se divide en cuatro piezas clave que se configuran mediante archivos independientes.

Primero, se necesita gestionar las contraseñas de forma segura; para ello se usa un "Secreto" en Kubernetes que guarda la clave del usuario *root* de la base de datos de forma oculta (codificada en Base64). Después, entra en juego la persistencia: como los contenedores en Kubernetes son efímeros (si se reinician, se borra todo lo que tienen dentro), se configuran unas peticiones de volumen (llamadas PVC) para pedirle a Amazon dos discos duros externos de 10 GB. Uno guardará los datos puros del MySQL y el otro los archivos multimedia e instalación del WordPress.

Finalmente, se despliegan las dos aplicaciones. Por un lado, el contenedor de MySQL, que se queda protegido en la red interna del clúster y no tiene salida a internet. Por otro, el contenedor de WordPress, que se conecta a un balanceador de cargas (LoadBalancer) provisto por AWS para generarnos una URL pública desde la que podamos acceder a la web.

Antes de arrancar, el video menciona un detalle súper importante: para que Kubernetes sepa cómo pedirle esos discos duros a Amazon, el clúster debe tener instalado previamente un complemento (Add-on) llamado driver de EBS. Si esto no se ha configurado previamente en la consola de AWS, los volúmenes darán error y no se crearán.

A la hora de ponerlo a funcionar en la terminal, el orden en el que ejecutamos los comandos es vital para que las piezas encajen correctamente:

Primero, se aplica el archivo del secreto para que la contraseña exista antes de que la base de datos intente arrancar. · Luego, se solicitan los discos duros aplicando el archivo de volúmenes. Estos discos se quedan "en espera" hasta que las aplicaciones los reclamen. · A continuación, se levanta la base de datos MySQL, la cual se apropia automáticamente de su disco de 10 GB. · Por último, se lanza el WordPress. Este se conecta a la base de datos interna y le pide a AWS que nos levante el balanceador de cargas público.

Una vez que todo está lanzado, el profesor no se queda solo en ver que los contenedores están encendidos, sino que realiza varias comprobaciones de diagnóstico (troubleshooting) muy útiles para documentar:

Utiliza comandos como **kubectl get all** y **kubectl get pvc** para tener una vista general de los recursos y comprobar que los discos se han asignado bien. · Entra de forma interactiva dentro del contenedor de MySQL usando una consola bash para comprobar que, efectivamente, la base de datos de WordPress se ha creado. · Lanza un contenedor temporal de pruebas solo para hacer un "ping" y revisar los puertos, asegurándose de que el WordPress y el MySQL se comunican bien por la red interna. · Simula un fallo borrando la contraseña a propósito. Aquí nos muestra cómo usar comandos como **kubectl describe pod** y **kubectl logs** para investigar y descubrir por qué un contenedor se queda bloqueado al arrancar.

Para terminar, cuando el balanceador de cargas nos entrega la URL definitiva, basta con entrar desde el navegador para completar la instalación gráfica típica de WordPress (crear el administrador, ponerle nombre al blog, etc.). Y como buena práctica en la nube, el video concluye enseñando cómo destruir toda esta infraestructura de forma limpia, ejecutando el borrado de los archivos para eliminar los contenedores y liberar los discos duros, evitando así costes extra en AWS.

PROYECTO Y OBJETIVO GENERAL

Nuestro proyecto consiste en desplegar un clúster de Kubernetes en AWS utilizando Amazon EKS y demostrar un despliegue realista de una aplicación web con una arquitectura típica de empresa: WordPress como capa de aplicación y MySQL como capa de datos, con conectividad interna entre servicios, persistencia de datos y exposición externa mediante un balanceador de carga. Todo el contenido teórico que explicamos a continuación no es “teoría vacía”, sino el porqué de cada decisión técnica que aplicaremos dentro de nuestro despliegue.

CONCEPTOS FUNDAMENTALES DE KUBERNETES APLICADOS A NUESTRO PROYECTO

Contenedores e imágenes

Los contenedores empaquetan aplicaciones con sus dependencias en una unidad portable y consistente. Una imagen es el “molde” inmutable del contenedor y contiene todo lo necesario para que la aplicación arranque igual en cualquier host compatible.
Nosotros desplegaremos dos imágenes principales: una de WordPress y una de MySQL. Esto nos permite separar claramente roles y responsabilidades y demostrar que la aplicación y la base de datos evolucionan de forma independiente. También nos permite justificar que el despliegue es repetible y estándar porque siempre partimos de versiones concretas de imagen.

Pod como unidad mínima de ejecución

En Kubernetes, el contenedor no se ejecuta solo, sino dentro de un Pod. Un Pod es la unidad mínima que Kubernetes programa en un nodo y puede contener uno o más contenedores que comparten red y volumen dentro del mismo contexto.
Aplicación al proyecto: WordPress se ejecutará dentro de un Pod y MySQL dentro de otro Pod. Esto es importante porque los Pods son efímeros, pueden desaparecer y volver a aparecer con una IP distinta, y este comportamiento nos sirve para demostrar por qué necesitamos service discovery y almacenamiento persistente para que la arquitectura siga funcionando aunque los Pods cambien.

Nodos y clúster

Un clúster es el conjunto de nodos que ejecutan Kubernetes. Los nodos son máquinas físicas o virtuales que aportan CPU, RAM, red y disco para que los Pods puedan correr. Kubernetes programa Pods en nodos según recursos y reglas.
En AWS EKS tendremos un control plane gestionado por AWS y un grupo de worker nodes como instancias EC2 dentro de un Auto Scaling Group. Esto nos permite explicar que nuestros Pods de WordPress y MySQL se ejecutan sobre nodos que se pueden reemplazar y escalar, y que Kubernetes no depende de una sola máquina.

Control plane y reconciliación del estado

Kubernetes funciona con un modelo declarativo. Nosotros definimos manifests que describen el estado deseado y el control plane se encarga de comparar el estado deseado con el estado real y corregir diferencias. Este mecanismo es la base de la resiliencia y la automatización.
Cuando definimos un Deployment de WordPress y MySQL estamos diciendo que queremos que haya siempre Pods corriendo con esa configuración. Si un Pod cae, Kubernetes lo recrea automáticamente, y esta es una demostración muy potente del proyecto porque podemos provocar un fallo y enseñar cómo el sistema se recupera.

Deployment y ReplicaSet

Un Deployment define cómo desplegar y actualizar Pods y normalmente crea un ReplicaSet para mantener el número de réplicas deseado. Esto permite escalar horizontalmente y también hacer actualizaciones controladas.
Aplicación al proyecto: WordPress lo definiremos como Deployment porque queremos poder escalar con réplicas si hace falta y porque queremos demostrar que si “matamos” un Pod, el sistema lo vuelve a levantar. Esto se puede explicar como disponibilidad y autohealing. En MySQL también podemos usar un Deployment para la demo, pero dejando claro que en entornos grandes a menudo se usa un patrón más específico para bases de datos. Lo que nos interesa en el proyecto es la persistencia y la conectividad interna.

Namespace como segmentación lógica

Los namespaces son particiones lógicas dentro del clúster que permiten separar aplicaciones y recursos y gestionar permisos y organización.
Aplicación al proyecto: Podemos trabajar en el namespace por defecto si el proyecto es simple, pero conceptualmente explicaremos que todo nuestro WordPress, MySQL y servicios forman parte de un mismo espacio lógico de proyecto, y que en una empresa real se podría usar un namespace dedicado para entorno de producción o desarrollo.

RED EN KUBERNETES Y CONECTIVIDAD ENTRE COMPONENTES DEL PROYECTO

Service discovery y por qué no usamos IPs de Pod

Las IPs de los Pods cambian cuando los Pods se recrean. Por eso Kubernetes introduce Services que ofrecen una IP estable y, sobre todo, un nombre DNS estable dentro del clúster. CoreDNS resuelve estos nombres y hace que las aplicaciones se puedan encontrar de forma fiable.
Aplicación al proyecto: WordPress debe conectarse a MySQL y lo hará apuntando al Service interno de MySQL, no a la IP del Pod. Esto es una prueba clara de que estamos aplicando arquitectura cloud correcta porque desacoplamos el direccionamiento de la vida efímera de los Pods y demostramos que la comunicación interna es robusta.

TIPOS DE SERVICIOS Y QUÉ UTILIZAREMOS EN EL PROYECTO

ClusterIP como servicio interno

ClusterIP expone un servicio solo dentro del clúster. Es ideal para backends como bases de datos porque no deben estar accesibles desde internet.
MySQL se expondrá con un Service de tipo ClusterIP. Esto permite que WordPress llegue a él internamente y, al mismo tiempo, mantenemos la base de datos protegida del exterior porque el proyecto quiere demostrar seguridad y buenas prácticas, no un diseño inseguro.

NodePort como forma de exponer pero menos profesional

NodePort abre un puerto alto en cada nodo y permite acceso externo si la red y los security groups lo permiten. Es funcional, pero obliga a gestionar puertos y reglas manualmente y no escala tan bien a nivel de arquitectura cloud moderna.
Aplicación al proyecto: Podemos mencionar NodePort como concepto porque lo hemos visto en los vídeos y porque sirve para comparar opciones, pero en nuestro proyecto lo usaremos solo como teoría o como prueba puntual si queremos demostrar alternativas. Nuestra opción principal será LoadBalancer porque es la que se ajusta a la nube de AWS.

LoadBalancer como entrada profesional a la aplicación

LoadBalancer crea un punto de entrada externo gestionado que en entornos cloud integra un balanceador real del proveedor. Esto da una URL pública y puede distribuir tráfico hacia los Pods.
WordPress se expondrá con un Service de tipo LoadBalancer porque queremos que el usuario acceda a la web desde internet de forma limpia y profesional y queremos demostrar la integración de EKS con los balanceadores de AWS. Esto nos permite enseñar un flujo real: el usuario entra por un DNS público y llega a la capa de aplicación sin saber nada del clúster.

PERSISTENCIA Y DATOS EN KUBERNETES ORIENTADO AL PROYECTO WORDPRESS MYSQL

Por qué la persistencia es obligatoria en nuestro caso

Los Pods son efímeros. Si una aplicación guarda datos solo dentro del filesystem del contenedor, esos datos desaparecen cuando el Pod se recrea. Por eso Kubernetes introduce Persistent Volumes y Persistent Volume Claims, que permiten montar almacenamiento persistente dentro de los Pods.
WordPress y MySQL necesitan persistencia sí o sí. MySQL guarda tablas e índices y WordPress guarda archivos, configuración y plugins. Por eso en el proyecto crearemos PVC para MySQL y PVC para WordPress. Esto nos permite demostrar que podemos reiniciar Pods o eliminarlos y la web continúa exactamente igual porque los datos están fuera del Pod en un volumen persistente.

EBS como backend de volúmenes en AWS

En AWS, un backend típico de persistencia para Kubernetes es EBS, que son volúmenes de bloque persistente asociados a instancias y gestionados por AWS. Kubernetes accede mediante drivers CSI y clases de almacenamiento.
Nosotros utilizaremos EBS como almacenamiento persistente porque es coherente con lo que hemos practicado y porque es fácil de justificar como integración cloud nativa. Además, nos permite demostrar que cuando hacemos un PVC de 10 GB, por ejemplo, AWS crea un volumen real que podemos ver desde la consola, y este punto ayuda mucho a la documentación porque conecta claramente Kubernetes con servicios AWS.

EFS como alternativa teórica y cuándo tendría sentido

EFS es un filesystem compartido tipo NFS que permite montaje simultáneo desde múltiples Pods y nodos y es útil cuando necesitamos shared storage real.
Nuestro foco es WordPress y MySQL con EBS porque es suficiente para demostrar persistencia e integración con AWS. Aun así, podemos mencionar EFS como alternativa teórica porque en algunos casos WordPress en alta disponibilidad puede necesitar almacenamiento compartido para archivos y EFS podría ser una opción. Lo importante es dejar claro que elegimos EBS porque encaja con la demo y los objetivos del proyecto y porque simplifica la explicación de PV y PVC.

SEGURIDAD Y CONFIGURACIÓN CON SECRETS EN EL PROYECTO

Secrets y por qué no ponemos contraseñas en plano

Kubernetes Secrets permite almacenar datos sensibles como contraseñas, tokens o claves e inyectarlos a Pods como variables de entorno o archivos. Aunque muchas veces van en base64, eso no es cifrado fuerte, pero sí un mecanismo estándar para separar configuración sensible de los manifests.
La contraseña de root de MySQL y la contraseña que WordPress usa para conectarse a MySQL no las pondremos en texto plano dentro del deployment, sino que las definiremos en un Secret y el Pod de MySQL las recibirá vía env desde el Secret. Este detalle es muy importante para demostrar buenas prácticas en entornos cloud y para explicar que la configuración se debe gestionar de forma segura y mantenible.

OBSERVABILIDAD Y TROUBLESHOOTING QUE USAREMOS

Kubectl como herramienta principal de operación

Kubectl es la herramienta cliente para interactuar con la API de Kubernetes. Permite aplicar manifests, consultar recursos y ver el estado de los objetos.
Aplicación al proyecto: Usaremos kubectl para aplicar los manifests de Secrets, PVC, Deployments y Services y también para validar que todo está en estado correcto. Esta parte será parte de la demostración porque es lo que nos permite probar la conectividad y ver que el sistema está desplegado.

Describe y logs como diagnóstico

Kubectl describe muestra información detallada de un recurso y especialmente los events que explican por qué un Pod no arranca o por qué hay errores. kubectl logs muestra los logs de un contenedor y sirve para entender qué está haciendo la aplicación.
Aplicación al proyecto: Podemos simular errores como un Secret mal definido o una key incorrecta y demostrar que con describe vemos el error y lo corregimos. Esto aporta valor porque muestra que no es solo montar, sino saber operar y resolver problemas.

EKS CLOUD Y HERRAMIENTAS AWS QUE APORTAN CONTEXTO REAL AL PROYECTO

Credenciales temporales y Cloud9

En AWS Academy, las credenciales son temporales y hay que actualizarlas para que herramientas como aws cli puedan autenticarse. Cloud9 es un entorno de desarrollo en la nube que facilita tener una terminal y herramientas dentro de AWS.
Aplicación al proyecto: En la documentación explicaremos que antes de trabajar con el clúster debemos asegurar credenciales correctas y configurar aws cli y kubectl para apuntar al clúster EKS. Esto forma parte de la “realidad cloud” y es un punto que nos diferencia de un ejercicio local porque dependemos de sesiones y autenticación en un entorno gestionado.

Kubeconfig y contexto del clúster

Kubeconfig contiene la configuración para conectarse a un clúster concreto y el contexto define a qué clúster y usuario apuntarán los comandos.
Aplicación al proyecto: Antes de desplegar WordPress y MySQL tenemos que tener el contexto correcto del clúster del proyecto y lo validaremos con comandos como get nodes, porque si el contexto es erróneo desplegaríamos en otro entorno o no tendríamos permisos. Esto es un paso básico, pero muy importante para documentar bien el flujo de trabajo.

Addons y drivers CSI para EBS

Para que Kubernetes pueda crear volúmenes persistentes en un proveedor hacen falta drivers de almacenamiento como el CSI de EBS. En EKS, esto a menudo se gestiona vía addons.
Aplicación al proyecto: Explicaremos que activamos o instalamos el driver necesario para que los PVC se puedan materializar en volúmenes EBS reales. Sin esto, el proyecto fallaría porque WordPress y MySQL no tendrían persistencia. Esto refuerza el mensaje de que la arquitectura cloud depende de integraciones entre capas.

FLUJO DE DESPLIEGUE ORIENTADO A LA MEMORIA DEL PROYECTO

Preparación del clúster

Antes de ejecutar manifests hay que tener un clúster operativo, herramientas instaladas y autenticación correcta.
Aplicación al proyecto: Crearemos o reutilizaremos el clúster EKS y validaremos nodos activos y contexto correcto. Después revisaremos addons necesarios y dejaremos el clúster preparado para almacenamiento y servicios.

Despliegue de la capa de datos MySQL

Primero se define el Secret con la contraseña, después el PVC para reservar persistencia, después el Deployment y el Service interno.
Aplicación al proyecto: Seguiremos este orden porque WordPress depende de MySQL y porque queremos demostrar buenas prácticas. Dejaremos MySQL como servicio interno con ClusterIP y validaremos que está accesible dentro del clúster.

Despliegue de la capa de aplicación WordPress

WordPress necesita variables de entorno para conectarse a MySQL, necesita persistencia y un servicio externo para recibir tráfico.
Aplicación al proyecto: Crearemos PVC para WordPress, desplegaremos el Deployment y crearemos el Service de tipo LoadBalancer. Esperaremos a que AWS aprovisione el balanceador y accederemos a la URL para completar el asistente de configuración de WordPress. Esto será la prueba final de éxito del proyecto.

Validaciones y demostraciones que aportan valor

Una arquitectura cloud no se demuestra solo “viendo la web”, sino demostrando propiedades como resiliencia, observabilidad y persistencia.
Aplicación al proyecto: Haremos pruebas de conectividad interna desde un Pod de herramientas hacia MySQL, comprobaremos puertos y DNS, y después simularemos la eliminación de un Pod para ver cómo el Deployment lo recrea. También comprobaremos que los datos no se pierden gracias a los volúmenes persistentes.

PROS/CONTRAS DEL ENFOQUE Y CÓMO LOS JUSTIFICAMOS EN EL PROYECTO

Ventajas principales

Kubernetes aporta automatización, resiliencia y capacidad de escalar y administrar aplicaciones de manera declarativa, repetible, y EKS aporta gestión del control plane e integración cloud.
A nosotros nos aporta un marco perfecto para demostrar prácticas reales como separar servicios, usar DNS interno, exponer por LoadBalancer y persistir datos con EBS. Esto da una arquitectura que se parece mucho a entornos profesionales y nos permite justificar el proyecto como “cloud ready”.

Limitaciones y decisiones conscientes

Kubernetes es complejo y no siempre es necesario para proyectos pequeños, y las bases de datos en contenedores tienen consideraciones especiales en producción.
Lo reconocemos y lo explicamos porque nuestro objetivo es aprendizaje y demostración de arquitectura en la nube, no hacer un producto final comercial. Por eso el valor está en mostrar el pipeline de despliegue, el control de recursos, la persistencia y la conectividad, más que en optimizaciones avanzadas.

CONCLUSIÓN CON ENFOQUE TOTAL AL PROYECTO

Este proyecto utiliza Kubernetes no como teoría, sino como herramienta para construir una arquitectura cloud real en AWS EKS, donde un WordPress y un MySQL funcionan como servicios separados comunicándose internamente con service discovery y DNS, exponiendo el front-end con un Load Balancer, manteniendo datos persistentes con volúmenes EBS gestionados mediante PVC y protegiendo credenciales con Secrets. El objetivo final es demostrar con pruebas claras que la plataforma es resiliente, que los Pods son efímeros pero la aplicación no pierde datos, que la conectividad es estable y que el acceso externo es profesional. Este equilibrio entre teoría y aplicación es justamente lo que queremos reflejar en la documentación y lo que hará que el proyecto parezca un despliegue cloud completo y coherente.

FullStack Deploy EKS

¿Qué es un FullStack Deploy EKS?

Un FullStack Deploy EKS Es el proceso de desplegar una aplicación completa (frontend, backend y servicios asociados) dentro de un clúster de Kubernetes gestionado por AWS EKS (Elastic Kubernetes Service).

En la práctica, implica unir tres mundos:

1. La aplicación *full-stack*

Incluye:

- Frontend
- Backend
- Servicios adicionales

Cada componente suele empaquetarse en contenedores Docker.

2. La infraestructura en EKS

EKS proporciona:

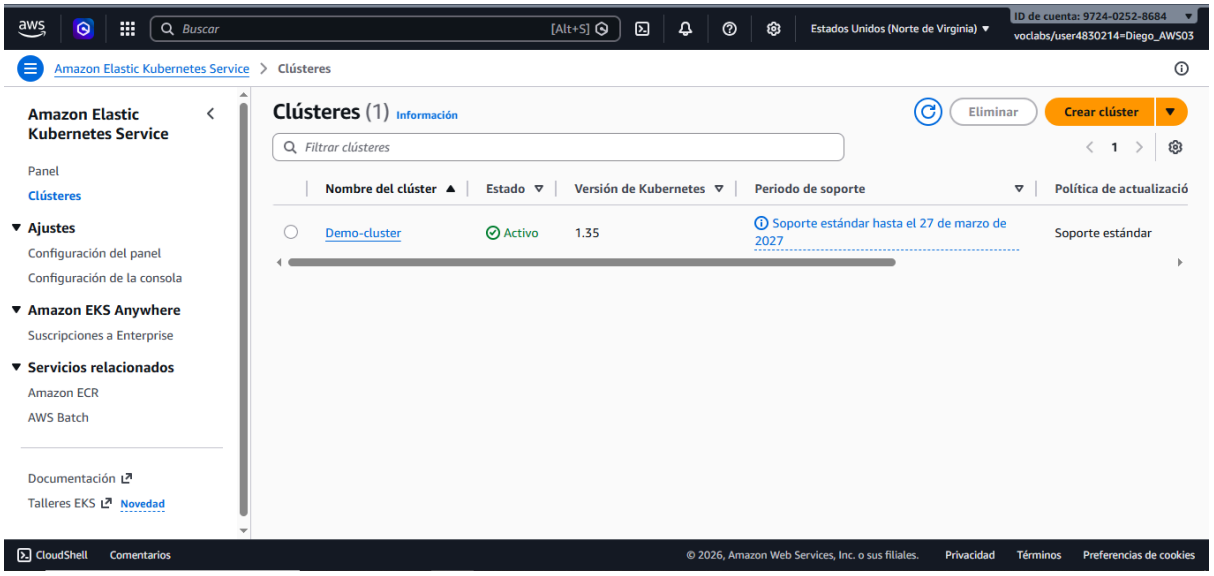
- Un clúster Kubernetes administrado por AWS
- Nodos de cómputo (EC2)

Esto permite ejecutar contenedores de forma escalable y segura.

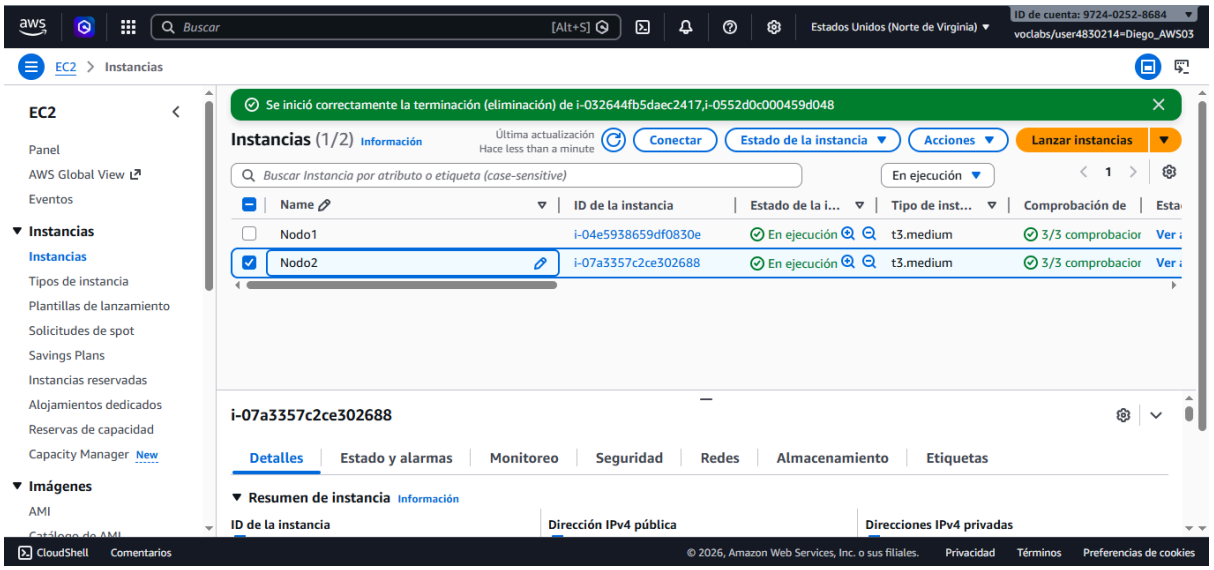
3. El despliegue (*deploy*)

El proceso típico incluye:

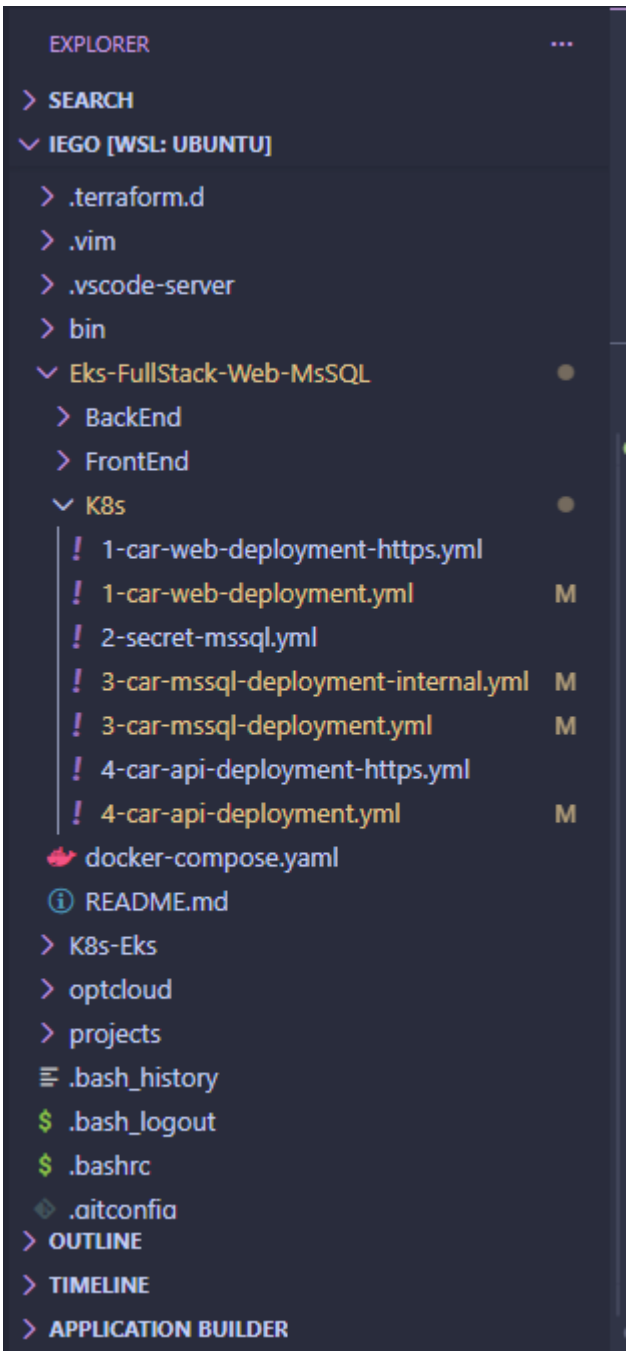
- Crear imágenes Docker del frontend y backend
- Subirlas a un registro (ECR)
- Definir manifiestos de Kubernetes (Deployments, Services, Ingress…)
- Aplicarlos al clúster EKS
- Automatizar todo con CI/CD (GitHub Actions, GitLab CI, ArgoCD…)



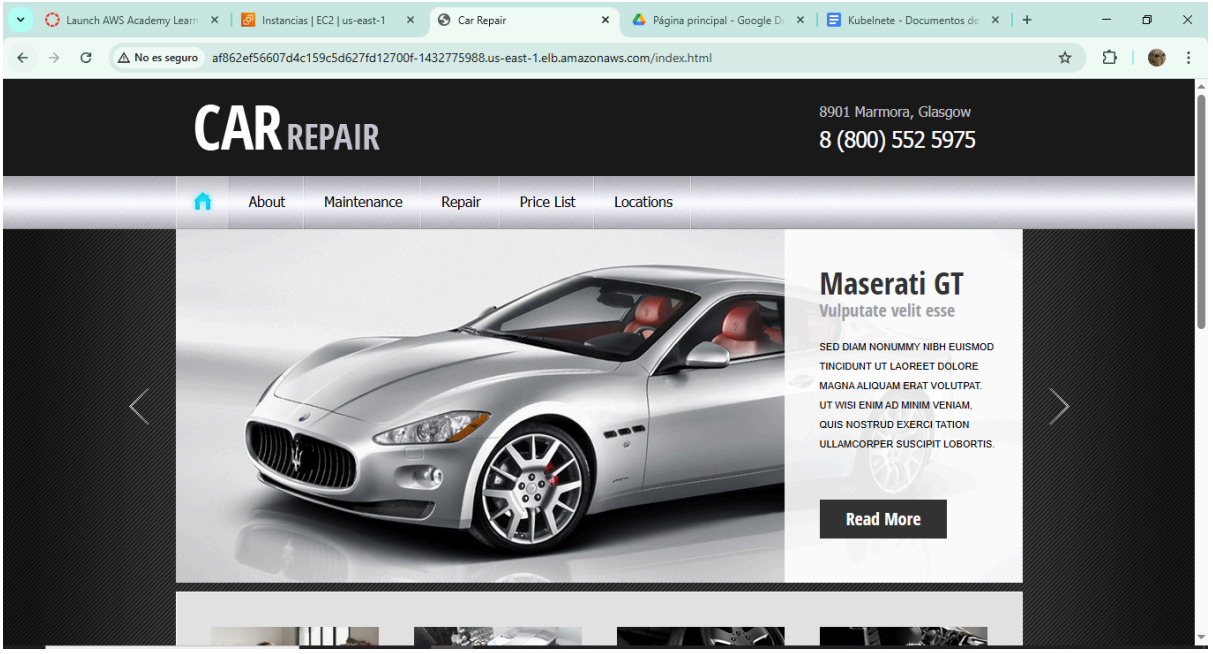
Primero desplegamos el Cluster.



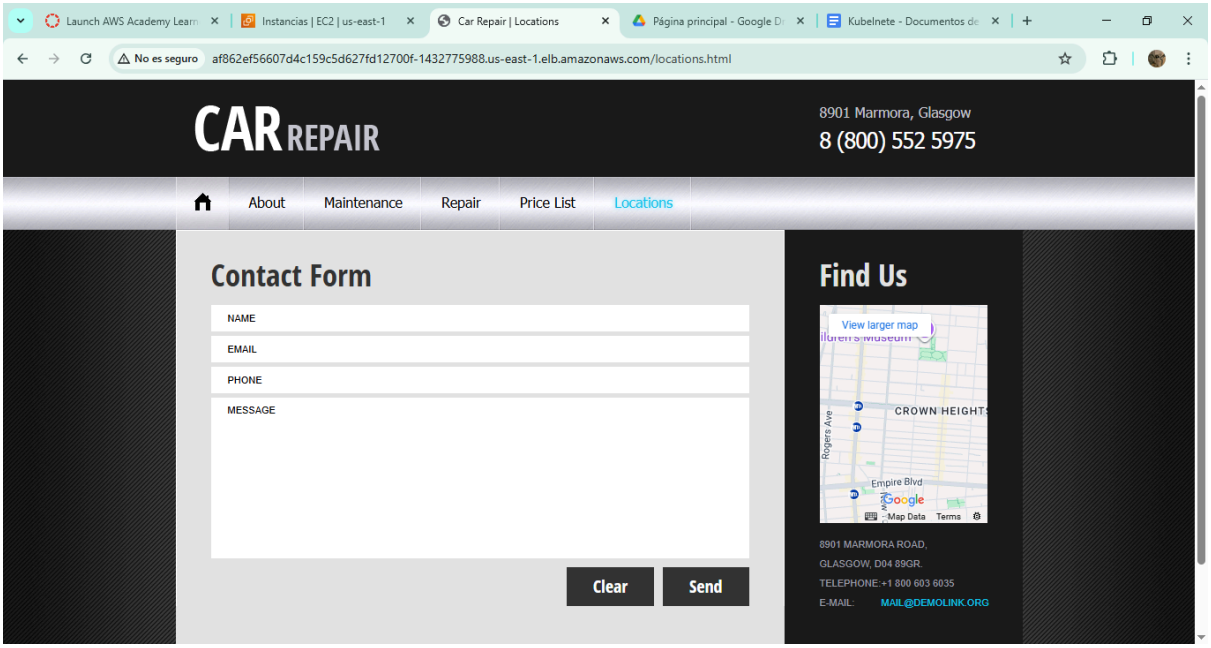
El Cluster como lo hemos configurado despliega 2 instancias de EC2.



Luego clonamos el repositorio de GitHub donde están los archivos que vamos a usar. Una vez los tenemos, simplemente los arrancamos con el comando kubectl apply -f y el nombre del archivo, y lo hacemos con los 4 archivos, cada uno sirve para uno caso, frontend, backend, base de datos, etc.



Y si lo hemos hecho bien ya tenemos la página hecha con balanceadores de carga, base de datos, etc.



Aquí está el formulario que va directo a la base de datos.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                                                    bash - iego

iego@DESKTOP-JITRG11:~$ kubectl get all
NAME                                    READY   STATUS    RESTARTS        AGE
pod/car-api-deploy-7fc4b98df-nvd24      1/1     Running   1 (3h53m ago)   3h56m
pod/car-web-deploy-c55dc5c9c-pb98t      1/1     Running   0               3h56m
pod/sql-server2022-69fdd99fc7-dc6km     1/1     Running   0               3h56m

NAME                      TYPE           CLUSTER-IP      EXTERNAL-IP                                                                    PORT(S)
      AGE
service/car-api-svc       LoadBalancer   10.100.37.72    ae0e6944ae8f64b449be7893a9e9ba01-240447521.us-east-1.elb.amazonaws.com    80:31473/T
CP    2d23h
service/car-web-svc       LoadBalancer   10.100.60.100   af862ef56607d4c159c5d627fd12700f-1432775988.us-east-1.elb.amazonaws.com   80:30263/T
CP    2d23h
service/kubernetes        ClusterIP      10.100.0.1      <none>                                                                         443/TCP
      7d23h
service/sql-server2022    LoadBalancer   10.100.87.115   a31b30df0533b4d12ab70897077456ab-845818869.us-east-1.elb.amazonaws.com    1433:30942
/TCP  2d23h

NAME                              READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/car-api-deploy    1/1     1            1           2d23h
deployment.apps/car-web-deploy    1/1     1            1           2d23h
deployment.apps/sql-server2022    1/1     1            1           2d23h

NAME                                         DESIRED   CURRENT   READY   AGE
replicaset.apps/car-api-deploy-7fc4b98df     1         1         1       2d23h
replicaset.apps/car-web-deploy-c55dc5c9c     1         1         1       2d23h
replicaset.apps/sql-server2022-69fdd99fc7    1         1         1       2d23h
iego@DESKTOP-JITRG11:~$ kubectl apply -f 3-car-mssql-deployment.yml

0   AWS: profile:default                                                                                   Finish Setup
```

Y con este comando podemos ver todo lo que tenemos vinculado a Kubernetes, podemos ver que tenemos los servicios, con balanceadores de carga, los desplazamientos y la base de datos, todo corriendo, activo, nos dice la IP y el puerto donde lo tenemos y todo.

WEBGRAFÍA

Lista de videos utilizada para la investigación:

https://docs.google.com/document/d/1LoyjUfhU371nZ0chLb0cXBkGDyAh_Dq8pclkKU4d4Ss/edit?usp=sharing

https://docs.aws.amazon.com/es_es/eks/latest/userguide/what-is-eks.html